Gradient Compression for Federated Learning: A Benchmark on Channel Estimation

Wageesha Lenmini
Department of Electronic and Telecommunication Engineering, University of Moratuwa
https://github.com/wageeshalenmini/fl-compression-bench

Abstract

In federated learning the uplink from devices to the server is the main bottleneck, and gradient compression is the standard remedy. This report implements five compression-based training methods from the literature (distributed compressed gradient descent with Rand-k and with Top-k, DIANA, classic error feedback, and EF21) in a tested simulator with exact bit accounting, and compares them on a task from wireless communications: federated training of a downlink channel estimator for a 64-antenna base station serving 20 users with different channel statistics. The model is linear, so its optimum is known in closed form and every run has an exact reference. At equal uplink budget and with an identical stepsize search for every method, EF21 with Top-k reaches within 0.1 dB of the best achievable error using about 16 times fewer uploaded bits than uncompressed gradient descent; DIANA and classic error feedback need 4 to 5 times fewer; plain Top-k compression stalls 1.3 to 3.5 dB short of the optimum. As the users' data becomes more different, classic error feedback and plain Rand-k compression degrade while EF21 and DIANA do not, in agreement with what their analyses assume. The stepsize permitted by the EF21 theorem is 88 to 462 times smaller than the best stepsize found. The study is a benchmark of existing methods on a new task and proposes no new method.

1 Introduction

Federated learning trains one shared model on data that remains on the devices that produced it [1, 2]. In each round a device computes an update from its local data and uploads it to a server, which aggregates the updates and broadcasts a new model. The update is as large as the model, and the uplink of a mobile device is slow, so the number of uploaded bits is the natural measure of cost.

Compression reduces this cost by sending only part of each update, for example a random subset of its coordinates or the coordinates of largest magnitude [2, 3]. Compression distorts the update, and a line of work in distributed optimization studies how to keep training convergent despite the distortion: DIANA compresses gradient differences [4], error feedback stores what was not sent [5, 6, 7], and EF21 maintains a compressed estimate of each local gradient [8]. These methods come with convergence theory, and their experiments are usually run on standard classification datasets.

This report asks how these methods compare on a problem from wireless communications, where the federated setting arises naturally: channel estimation, in which each handset holds received pilot signals that reflect its own propagation environment. The specific questions are:

1. For a federated channel estimator, which compression method reaches a given estimation accuracy with the fewest uploaded bits?
2. How does the comparison change as the users' channel statistics become more different?
3. How does the stepsize permitted by theory compare with the stepsize that works best?

Scope. The work is an implementation and an empirical comparison of published methods. Federated learning has been applied to channel estimation before [10]; the contribution here is the controlled comparison of compression methods on that task, with a model whose optimum is known exactly, equal bit budgets, equal tuning effort, and a test suite that checks each method against known properties.

2 Background

2.1 Problem

There are n devices. Device i holds m_i data points and has the local loss $f_i : \mathbb{R}^d \to \mathbb{R}$, the average loss of the model on its data. The goal is

$$\min_{x\in\mathbb{R}^d} f(x) = \sum_{i=1}^{n} p_i f_i(x), \qquad p_i = \frac{m_i}{\sum_j m_j}, \qquad \nabla f(x) = \sum_{i=1}^{n} p_i \nabla f_i(x). \tag{1}$$

No similarity between the f_i is assumed. In particular, at a minimizer $x^\star$ the average gradient is zero but the individual gradients $\nabla f_i(x^\star)$ are in general not zero. This fact drives most of the results below. All experiments use full local gradients and full participation, so the only randomness is in the compressors.

2.2 Compressors

A compressor $\mathcal{C}$ maps a vector to one that is cheaper to transmit. Two classes are used in the theory. An *unbiased* compressor satisfies

$$\mathbb{E}[\mathcal{C}(v)] = v, \qquad \mathbb{E}\left\|\mathcal{C}(v) - v\right\|^2 \le \omega \left\|v\right\|^2, \tag{2}$$

and a *contractive* compressor satisfies, for some $0 < \alpha \le 1$,

$$\mathbb{E}\left\|\mathcal{C}(v) - v\right\|^2 \le (1-\alpha)\left\|v\right\|^2. \tag{3}$$

Rand-k keeps k coordinates chosen uniformly at random and multiplies them by d/k. It is unbiased with $\omega = d/k - 1$, with equality. The device and the server generate the mask from a shared seed [2], so no positions are transmitted. *Top-k* keeps the k coordinates of largest magnitude without scaling. It is deterministic, biased, and contractive with $\alpha = k/d$ [9]. Its positions depend on the data and must be transmitted.

Bit accounting. A real number costs 32 bits. One upload costs

$$\text{dense: } 32d, \qquad \text{Rand-}k\text{: } 32k + 32, \qquad \text{Top-}k\text{: } k\,(32 + \lceil \log_2 d \rceil). \tag{4}$$

Only the uplink is counted, as in the papers this report follows.

2.3 Methods

Let $g_i^t = \nabla f_i(x^t)$ and let $\gamma > 0$ be the stepsize. Each method is implemented as a device step, which sees only the device's own gradient and state, and a server step, which sees only messages.

GD. Devices send g_i^t and the server sets $x^{t+1} = x^t - \gamma \sum_i p_i g_i^t$.

DCGD. Devices send $\mathcal{C}(g_i^t)$ and the server sets $x^{t+1} = x^t - \gamma \sum_i p_i \mathcal{C}(g_i^t)$. This is compression with no correction.

DIANA [4]. Device i keeps a shift h_i (initially zero) and the server keeps $h = \sum_i p_i h_i$:

$$\Delta_i^t = \mathcal{C}\big(g_i^t - h_i^t\big), \qquad h_i^{t+1} = h_i^t + \alpha_h \Delta_i^t, \tag{5}$$

$$\hat{g}^t = h^t + \sum_i p_i \Delta_i^t, \qquad x^{t+1} = x^t - \gamma \hat{g}^t, \qquad h^{t+1} = h^t + \alpha_h \sum_i p_i \Delta_i^t. \tag{6}$$

It requires an unbiased compressor. The version used here has no momentum and no proximal step, with $\alpha_h = 1/(1+\omega)$.

Classic error feedback (EF) [5, 6, 7]. Device i keeps an error e_i (initially zero):

$$v_i^t = \mathcal{C}\big(e_i^t + \gamma g_i^t\big), \qquad e_i^{t+1} = e_i^t + \gamma g_i^t - v_i^t, \qquad x^{t+1} = x^t - \sum_i p_i v_i^t. \tag{7}$$

EF21 [8]. Device i keeps an estimate g_i of its own gradient and the server keeps their average g:

$$c_i^t = \mathcal{C}\big(\nabla f_i(x^t) - g_i^{t-1}\big), \qquad g_i^t = g_i^{t-1} + c_i^t, \qquad g^t = g^{t-1} + \sum_i p_i c_i^t, \qquad x^{t+1} = x^t - \gamma g^t, \tag{8}$$

with $g_i^{-1} = 0$, so that the first message is $\mathcal{C}(\nabla f_i(x^0))$. For L_i-smooth f_i and a contractive compressor, the EF21 theorem guarantees an $O(1/T)$ rate for smooth nonconvex problems when

$$\gamma \le \frac{1}{L + \tilde{L}\sqrt{\beta/\theta}}, \qquad \theta = 1 - \sqrt{1-\alpha}, \quad \beta = \frac{1-\alpha}{1-\sqrt{1-\alpha}}, \quad \tilde{L} = \Big(\tfrac{1}{n}\textstyle\sum_i L_i^2\Big)^{1/2}. \tag{9}$$

The paper states this for equal weights; with unequal p_i the implementation uses $\tilde{L} = (\sum_i p_i L_i^2)^{1/2}$, which is an extension made here and not a result of the paper. In the experiments all devices hold the same number of points, so the two coincide.

3 The channel estimation task

3.1 System and channel model

A base station with $N = 64$ antennas in a uniform linear array with half-wavelength spacing transmits $L_p = 32$ pilot symbols. A single-antenna user receives

$$y = Ph + w, \qquad y \in \mathbb{C}^{L_p},\ h \in \mathbb{C}^N,\ w \sim \mathcal{CN}(0, \sigma^2 I), \tag{10}$$

where the rows of the pilot matrix $P \in \mathbb{C}^{L_p \times N}$ are random QPSK sequences with unit norm. The user must estimate h from y. Since $L_p < N$, this is impossible without prior knowledge of how channels are distributed.

Channels follow a geometric multipath model,

$$h = \sqrt{\tfrac{N}{n_p}} \sum_{\ell=1}^{n_p} \alpha_\ell\, a(\theta_\ell), \qquad \alpha_\ell \sim \mathcal{CN}(0,1), \qquad [a(\theta)]_k = \tfrac{1}{\sqrt{N}} e^{j\pi k \sin\theta}, \tag{11}$$

with $n_p = 3$ paths, so that $\mathbb{E}\,\|h\|^2 = N$. With unit-norm pilot rows the signal-to-noise ratio is $\mathrm{SNR} = 1/\sigma^2$. The angles of user u are uniform on a sector of width 10° centred at c_u, and the centres of the $n = 20$ users are evenly spread over a range of 30° by default. Each user therefore has its own covariance $R_u = \mathbb{E}[hh^{\mathsf{H}}]$, and the *spread* of the centres controls how different the users are; a spread of zero makes all users statistically identical.

3.2 Estimators

For a linear estimator $\hat{h} = Wy$ and channel covariance R, the mean squared error is exactly

$$\mathbb{E}\,\|Wy - h\|^2 = \operatorname{tr}\big((WP - I)R(WP - I)^{\mathsf{H}}\big) + \sigma^2\,\|W\|_F^2\,. \tag{12}$$

Three classical references are used. *Least squares* takes $W = P^{+}$, the pseudo-inverse, and uses no prior knowledge. *Shared LMMSE* takes $W = \bar{R}P^{\mathsf{H}}(P\bar{R}P^{\mathsf{H}} + \sigma^2 I)^{-1}$ with the average covariance $\bar{R} = \frac{1}{n}\sum_u R_u$; it is the best single linear estimator for all users [11]. *Per-user LMMSE* uses each user's own R_u and is a lower bound that no shared estimator can reach. Errors are reported as normalized mean squared error, $\text{NMSE} = \mathbb{E}\left\|\hat{h} - h\right\|^2 / \mathbb{E}\,\|h\|^2$, in dB.

3.3 The federated learning problem

The model is one matrix W shared by all users, trained on pairs (y, h) with the loss

$$f_i(W) = \frac{1}{2m_i}\sum_{j=1}^{m_i}\|Wy_{ij} - h_{ij}\|^2\,. \tag{13}$$

Complex vectors are stored as their real and imaginary parts, so $W \in \mathbb{R}^{2N\times 2L_p}$ and $d = 128 \times 64 = 8192$. Each user holds $m_i = 500$ training pairs, and 1000 further pairs per user form the test set.

This model is chosen deliberately. The problem is a convex quadratic with a closed-form minimizer, and as the amount of data grows that minimizer tends to the shared LMMSE estimator. Every training run therefore has an exact target: the error of the exact minimizer of the training loss, evaluated on the test set, called the *best reachable* error below.

Table 1: Estimation error (NMSE in dB) of the references and of the shared estimator trained by uncompressed federated gradient descent for 600 rounds. Default system.

SNR (dB)	Least squares	Shared LMMSE	Per-user LMMSE	Federated GD
−10	10.07	−0.57	−1.65	−0.55
−5	5.51	−1.45	−3.71	−1.43
0	1.66	−3.05	−6.87	−3.03
5	−0.95	−5.38	−10.82	−5.36
10	−2.26	−8.30	−15.19	−8.28
15	−2.77	−11.78	−19.74	−11.77
20	−2.94	−15.79	−24.39	−15.80

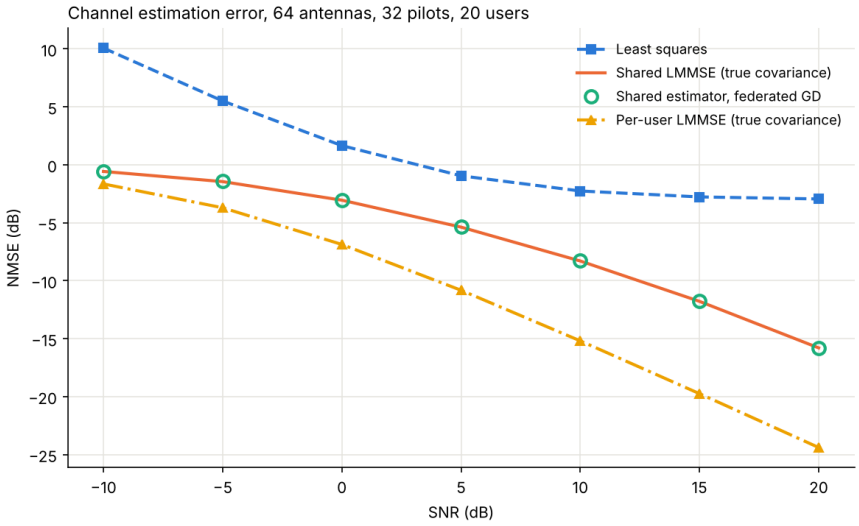


Figure 1: Estimation error against SNR. The federated estimator, trained on samples only, coincides with the shared LMMSE estimator built from the true covariance.

Table 1 and Figure 1 show that the task is meaningful. Least squares fails, since 32 pilots cannot determine 64 unknowns. The federated estimator matches the shared LMMSE estimator to within 0.02 dB at every SNR and gains 6 dB over least squares at 10 dB SNR. The remaining gap to per-user LMMSE, about 7 dB at 10 dB SNR, is the price of using one linear model for all users.

4 Implementation and verification

The simulator is written in NumPy. A training method never receives data, only gradients and messages, which keeps the code faithful to what a device and a server can each see. The repository contains 118 unit tests. The ones that matter most for trusting the results are:

- federated gradient descent equals gradient descent on the pooled data to ten decimal places, for equal and unequal device sizes;
- Rand-k is exactly unbiased with variance exactly $(d/k - 1)\,\|v\|^2$, verified by enumerating all masks on a small vector, and Top-k satisfies its contraction bound with equality for equal magnitudes;
- with no compression ($k = d$) every method reproduces gradient descent;
- DCGD with Top-1 diverges on the three-function example of [9], and one step matches the hand calculation;
- DIANA converges to the exact least squares solution where DCGD stalls, and its shifts converge to the local gradients at the solution;
- for classic error feedback, the virtual iterate $x^t - \sum_i p_i e_i^t$ follows exact gradient descent;
- for EF21, the server estimate equals $\sum_i p_i g_i$ in every round, and the measured average of $\left\|\nabla f(x^t)\right\|^2$ is below the bound of the paper's theorem at stepsize (9);
- formula (12) matches simulation for least squares and LMMSE, and LMMSE is optimal against random perturbations.

Table 2: Logistic regression on a9a ($d = 123$, 20 devices), at 100,000 uploaded bits per device. L is an upper bound on the smoothness constant.

Method	Rounds	Best stepsize	Final $\|\nabla f\|^2$
GD (no compression)	25	$2/L$	4.6×10^{-8}
DCGD + Top-1	2564	$2/L$	4.4×10^{-2}
EF + Top-1	2564	$0.0156/L$	4.0×10^{-5}
EF21 + Top-1	2564	$2/L$	4.1×10^{-33}
DCGD + Rand-2	1041	$0.0156/L$	1.9×10^{-3}
DIANA + Rand-2	1041	$0.125/L$	2.4×10^{-10}

5 Experimental protocol

All experiments use the default system at 10 dB SNR unless stated otherwise. The comparison follows four rules.

- **Equal budget.** Every method may upload $30 \times 32d = 7.86$ Mbit per device, the cost of 30 uncompressed rounds. Methods that send less per round run for more rounds.
- **Equal tuning.** Every method is run at stepsizes $\gamma = 2^j/L$ for $j = -7, \ldots, 2$, where L is the smoothness constant of f.
- **Target.** The target is to come within 0.1 dB of the best reachable test NMSE. For each method the stepsize kept is the one that reaches the target with the fewest bits; if none reaches it, the one with the lowest final error.
- **Randomness.** Methods using Rand-k are tuned on one seed and then averaged over three. Methods using Top-k are deterministic.

Compression levels are $k = 82$, 410 and 819, that is 1%, 5% and 10% of the coordinates. With $d = 8192$ a Top-k coordinate costs 45 bits and a Rand-k coordinate 32 bits.

6 Results

6.1 Bits needed at three compression levels

Table 3: Mbit uploaded per device to come within 0.1 dB of the best reachable NMSE (−8.28 dB), and final NMSE in dB in parentheses. Budget 7.86 Mbit.

Method	keep 1%	keep 5%	keep 10%
GD (no compression)	7.86 (−8.19)	7.86 (−8.19)	7.86 (−8.19)
DCGD + Top-k	not reached (−4.76)	not reached (−6.28)	not reached (−6.95)
EF + Top-k	1.64 (−8.20)	1.64 (−8.21)	1.95 (−8.19)
EF21 + Top-k	**0.50** (−8.28)	**0.63** (−8.28)	**0.77** (−8.28)
DCGD + Rand-k	7.27 (−8.19)	6.87 (−8.19)	6.43 (−8.19)
DIANA + Rand-k	1.73 (−8.28)	1.67 (−8.28)	1.86 (−8.28)

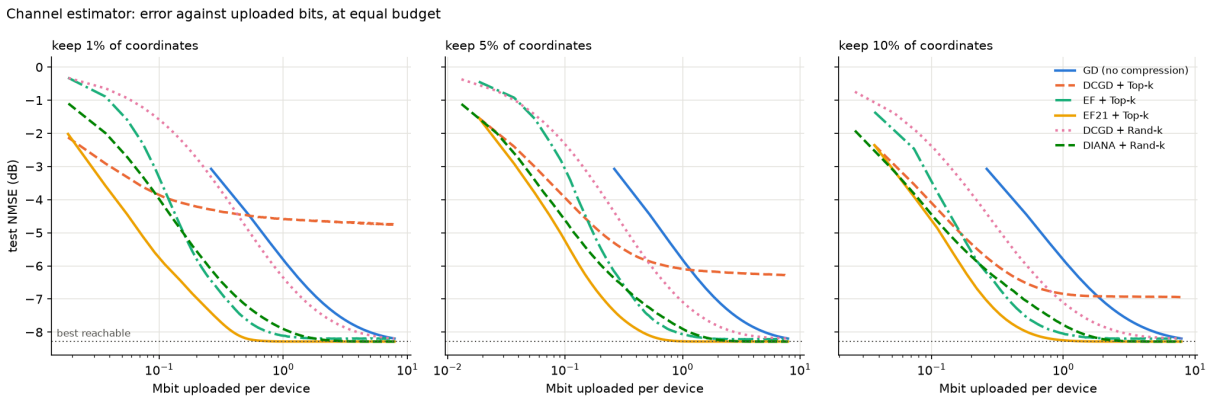


Figure 2: Test NMSE against uploaded bits per device, one panel per compression level. The dotted line is the best reachable error.

Table 3 and Figure 2 give the main result.

- EF21 with Top-k is the cheapest at every level. At 1% it needs 0.50 Mbit against 7.86 Mbit for uncompressed gradient descent, a factor of about 16.
- DIANA with Rand-k and classic error feedback with Top-k need 1.6 to 2.0 Mbit, a factor of 4 to 5 below gradient descent.
- DCGD with Top-k never reaches the target. It stalls at -4.76, -6.28 and -6.95 dB for 1%, 5% and 10%.
- DCGD with Rand-k reaches the target, but with 6.4 to 7.3 Mbit it saves almost nothing over gradient descent.
- For the three methods that work well, stronger compression is not worse per bit: the cost is lowest or nearly lowest at 1%.

6.2 Effect of data heterogeneity

Table 4: Mbit per device to the target as the users' sector centres are spread further apart, keeping 5% of the coordinates. The best reachable error changes with the spread, so methods should be compared within a column.

Spread of sector centres	0°	15°	30°	60°
Best reachable NMSE (dB)	−14.93	−11.16	−8.28	−4.86
GD (no compression)	not reached	7.08	7.86	5.77
DCGD + Top-k	not reached	not reached	not reached	not reached
EF + Top-k	**0.44**	**0.66**	1.64	2.31
EF21 + Top-k	1.35	0.83	**0.63**	**0.48**
DCGD + Rand-k	1.47	2.75	6.87	not reached
DIANA + Rand-k	1.97	1.71	1.67	1.49

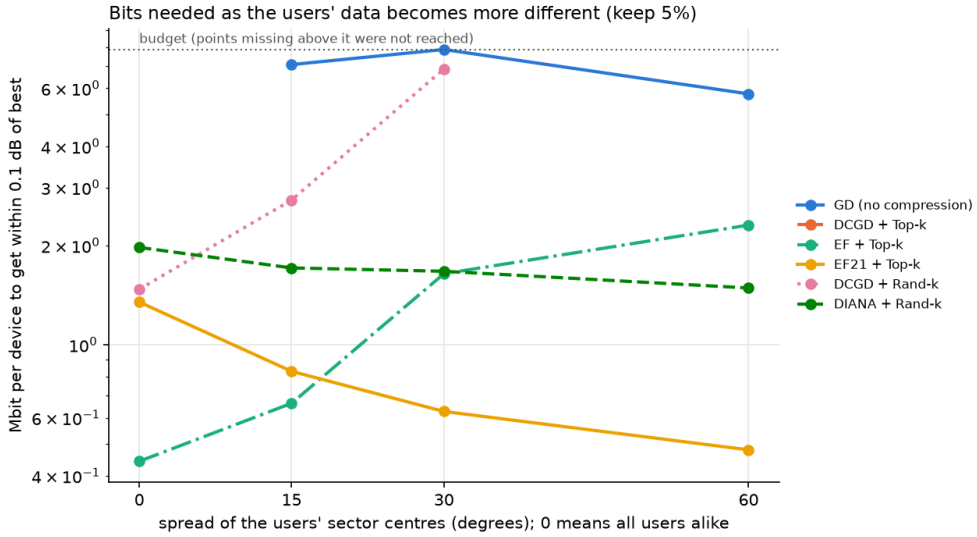


Figure 3: Bits needed to reach the target against the spread of the users' sector centres. A missing point means the target was not reached within the budget.

Table 4 and Figure 3 show how the ranking depends on how different the users are.

- When all users are alike (spread 0°), classic error feedback is the cheapest method and plain Rand-k compression works well.
- As the spread grows, both degrade. Classic error feedback goes from 0.44 to 2.31 Mbit, and its best stepsize falls from $2/L$ to $1/L$, $0.5/L$ and $0.25/L$. DCGD with Rand-k goes from 1.47 Mbit to not reaching the target.
- EF21 and DIANA do not degrade. DIANA stays between 1.5 and 2.0 Mbit and EF21 between 0.5 and 1.35 Mbit.

6.3 Theoretical and tuned stepsize for EF21

Table 5: EF21 with Top-k: stepsize permitted by (9) against the best stepsize found on the grid.

Kept	Theorem	Tuned	Ratio	NMSE at theorem's stepsize	Target reached?
1%	$0.0022/L$	$1/L$	462	−5.99 dB	no
5%	$0.0111/L$	$2/L$	181	−6.04 dB	no
10%	$0.0227/L$	$2/L$	88	−6.10 dB	no

Table 5 compares the stepsize allowed by the EF21 theorem with the best stepsize on the grid. The tuned stepsize is 88 to 462 times larger, and at the theorem's stepsize the method does not reach the target within the budget at any compression level. In the unit tests the method does satisfy the bound of its theorem at that stepsize, so the theorem holds; it is conservative for this problem.

7 Discussion

Why plain compression fails. At the solution $\sum_i p_i \nabla f_i(x^\star) = 0$, but the individual $\nabla f_i(x^\star)$ are not zero when the users' data differs. DCGD keeps compressing these nonzero vectors forever. With Top-k the compressed vectors $\mathcal{C}(\nabla f_i(x^\star))$ do not average to zero in general, so $x^\star$ is not a fixed point of the iteration and the method settles elsewhere. The offset shrinks as k grows, which

matches the stall levels of -4.76, -6.28 and -6.95 dB. With Rand-k the average is correct in expectation, but its variance, $\sum_i p_i^2(d/k-1)\,\|\nabla f_i(x)\|^2$, does not vanish at $x^\star$. A fixed stepsize then reaches only a neighbourhood whose size is proportional to γ, as the unit tests confirm, so reaching a tight target forces a small stepsize and many rounds. When the users are alike, $\nabla f_i(x^\star) \approx 0$ and the problem disappears, which is why DCGD with Rand-k works at spread 0° and fails at 60°.

Why DIANA and EF21 are unaffected. Both compress a difference that tends to zero. DIANA sends $\mathcal{C}(g_i - h_i)$ with $h_i \to \nabla f_i(x^\star)$, and EF21 sends $\mathcal{C}(\nabla f_i(x) - g_i)$ with $g_i \to \nabla f_i(x)$; both limits are verified in the test suite. The compression error therefore vanishes at the solution whatever the size of $\nabla f_i(x^\star)$, and neither analysis needs the users to be similar.

Why classic error feedback degrades. Its virtual iterate follows exact gradient descent, but the real iterate differs from it by the stored errors e_i, whose size scales with $\gamma\,\|\nabla f_i\|$. Since ∇f_i does not vanish at the solution under heterogeneity, the errors stay large unless γ is small. This is consistent with the observed halving of its best stepsize each time the spread increases, and with earlier analyses of error feedback needing bounded gradients or bounded dissimilarity [6, 7]. It is an interpretation of the measurements, not a proof.

Why the theoretical stepsize is so small. For small α, $\theta \approx \alpha/2$ and $\beta \approx 2/\alpha$, so $\sqrt{\beta/\theta} \approx 2/\alpha$ and (9) gives $\gamma \approx \alpha/(2\tilde{L})$: the permitted stepsize shrinks in proportion to k/d. The measured ratios of 462, 181 and 88 at $\alpha = 0.01$, 0.05 and 0.10 fall as α grows, in line with this. The constant $\alpha = k/d$ is the worst case for Top-k, attained when all entries have equal magnitude. A plausible explanation of the gap is that the vectors compressed in practice are far from that worst case, so that Top-k retains much more than a fraction k/d of their energy. This was not measured here and would be a natural follow-up.

An observation left open. In Table 4, EF21 needs fewer bits as the spread grows, and at spread 0° classic error feedback beats it by a factor of 3. The best reachable error and the conditioning of the problem both change with the spread, so numbers in different columns are not directly comparable, and this report does not offer an explanation for either effect.

8 Limitations

- **One model, convex.** The estimator is linear. It cannot beat the shared LMMSE estimator, and the results say nothing yet about nonconvex models such as neural network estimators.
- **One data draw and one SNR** for the main comparison. Top-k methods are single deterministic runs; Rand-k methods are averaged over three seeds.
- **Choices in the protocol.** The 0.1 dB tolerance, the budget, the stepsize grid and the rule for selecting the stepsize are choices. Different choices could reorder methods that are close, such as DIANA and classic error feedback.
- **Resolution.** Bits to target are read from about 400 logged points per run.
- **Idealized training.** Full local gradients and full participation are used. The true channel serves as the training label, which a handset does not have in practice. The uplink is error-free and the downlink is not counted.
- **Theory comparison.** Only the EF21 stepsize was taken from its theorem. The theoretical stepsizes of the other methods were not evaluated.
- **Literature check.** The a9a check agrees with the EF21 paper in behaviour only. The comparison with earlier work on federated channel estimation is limited to citing it.

9 Conclusion

On a federated channel estimation task with an exactly known optimum, the compression methods behave as their theory predicts, and the size of the effect is substantial. EF21 with Top-k reaches the target accuracy with about 16 times fewer uploaded bits than uncompressed training, and DIANA and classic error feedback with 4 to 5 times fewer. Plain compression is not a usable alternative when users differ: Top-k stalls and Rand-k saves nothing. Heterogeneity separates the methods clearly, hurting classic error feedback and plain compression while leaving EF21 and DIANA unaffected. The stepsize allowed by the EF21 theorem is two orders of magnitude below what works best here.

Natural extensions are a neural network estimator, to test whether the ranking survives nonconvexity and whether a nonlinear model can close part of the 7 dB gap to per-user LMMSE; minibatch gradients and partial participation; measuring how compressible the transmitted vectors are, to test the explanation offered for the stepsize gap; and evaluating the theoretical stepsizes of DIANA and classic error feedback.

References

[1] H. B. McMahan, E. Moore, D. Ramage, S. Hampson and B. Agüera y Arcas. Communication-efficient learning of deep networks from decentralized data. In *AISTATS*, 2017.

[2] J. Konečný, H. B. McMahan, F. X. Yu, P. Richtárik, A. T. Suresh and D. Bacon. Federated learning: strategies for improving communication efficiency. arXiv:1610.05492, 2016.

[3] D. Alistarh, D. Grubic, J. Li, R. Tomioka and M. Vojnovic. QSGD: communication-efficient SGD via gradient quantization and encoding. In *NeurIPS*, 2017.

[4] K. Mishchenko, E. Gorbunov, M. Takáč and P. Richtárik. Distributed learning with compressed gradient differences. arXiv:1901.09269, 2019.

[5] F. Seide, H. Fu, J. Droppo, G. Li and D. Yu. 1-bit stochastic gradient descent and its application to data-parallel distributed training of speech DNNs. In *Interspeech*, 2014.

[6] S. U. Stich, J.-B. Cordonnier and M. Jaggi. Sparsified SGD with memory. In *NeurIPS*, 2018.

[7] S. P. Karimireddy, Q. Rebjock, S. U. Stich and M. Jaggi. Error feedback fixes SignSGD and other gradient compression schemes. In *ICML*, 2019.

[8] P. Richtárik, I. Sokolov and I. Fatkhullin. EF21: a new, simpler, theoretically better, and practically faster error feedback. In *NeurIPS*, 2021. arXiv:2106.05203.

[9] A. Beznosikov, S. Horváth, P. Richtárik and M. Safaryan. On biased compression for distributed learning. *Journal of Machine Learning Research*, 24, 2023. arXiv:2002.12410.

[10] A. M. Elbir and S. Coleri. Federated learning for channel estimation in conventional and RIS-assisted massive MIMO. *IEEE Transactions on Wireless Communications*, 2022.

[11] S. M. Kay. *Fundamentals of Statistical Signal Processing: Estimation Theory.* Prentice Hall, 1993.

[12] C.-C. Chang and C.-J. Lin. LIBSVM: a library for support vector machines. *ACM Transactions on Intelligent Systems and Technology*, 2(3), 2011.